

COMP24011 Lab3:

Features for Estimating Autonomous Vehicle Poses

Riza Batista-Navarro and Francisco Lobo

Academic session: 2024-25

Introduction

In this exercise, you will investigate matching visual features in a series of images captured during the navigation of an autonomous vehicle (AV). These features are used in estimating poses (i.e., camera trajectories) based on a visual odometry algorithm. Let's start by introducing some terms and their definitions to help clarify concepts in autonomous robot navigation.

Odometry is the use of sensors to estimate a robot's change in position relative to a known position. *Visual odometry* (VO) is a specific type of odometry where only cameras are used as sensors, as opposed to using, e.g., global positioning system (GPS) sensors or light detection and ranging (LIDAR) sensors. It is based on the analysis of a sequence of camera images. *Simultaneous localisation and mapping* (SLAM) is a task whereby a robot needs to build a map of its current environment while at the same time trying to determine its position relative to that map.

In this exercise, you will explore a monocular (single-camera) VO solution to the [SLAM Evaluation 2012](#) challenge, which made use of the KITTI data set. However, it is worth noting that VO is limited in that it can only perform trajectory estimation after each pose, and hence trajectory optimisation is achieved only locally. In contrast, global optimisation is achieved through loop closure: the correction of the trajectory upon revisiting an already encountered location.

The original KITTI data set consists of 4541 frame images and occupies several gigabytes. To reduce disk quota and running time requirements, this lab exercise will use a simplified version of the data set that we refer to as MyKITTI. This contains only the first 501 frames of the original, but these still occupy close to 150 MB and so **cannot be included** in your GitLab repo. For this reason, you will need to [download MyKITTI.zip](#) from Blackboard and extract it.

👉 The examples in this manual assume that you extract `MyKITTI.zip` in your home directory, and hence that the path to the data set is `~/MyKITTI`. The command-line tool for this lab accepts an optional parameter allowing you to set a different path for the MyKITTI images.

In the `lab3` branch of your `COMP24011_2024` GitLab repo you will find a fully functional monocular VO system. The implementation provided to you builds upon another library available on [GitHub](#), which was ported to Python 3 and recent OpenCV 4 versions, and was significantly changed to add the functionality required for this lab exercise.

The VO system depends on the Python bindings for [OpenCV](#). These are already installed on the Kilburn lab machines. In any case, to use our VO system you should install the required Python module by issuing the command

```
$ pip install opencv-contrib-python
```

OpenCV is a comprehensive computer vision framework, for which extensive [documentation](#) and [tutorials](#) are available. It is very popular in both academic and industrial contexts, and thus benefits from active development. The continuous development of OpenCV has involved changes in data structures as well as changes in the API due to [patent issues](#). The lab code depends on OpenCV 4 functionality which has changed in recent years.

- 👉 As explained below, until you fully implement the tasks of this lab exercise the VO system **can behave differently** depending on the OpenCV version used to run it. The default Ubuntu image on Kilburn lab machines has two versions of OpenCV installed.

```
$ python3 --version
Python 3.10.12
$ python3 -c 'import cv2;print(cv2.__version__)'
4.5.4
$ python --version
Python 3.11.7
$ python -c 'import cv2;print(cv2.__version__)'
4.6.0
```

Using the above `pip` command on the lab machines will install an additional version (4.10.x).

In this exercise you will use OpenCV to complete computer vision tasks, which will involve getting familiar with its [2D Features Framework](#). The VO system for the lab uses the Scale-Invariant Feature Transform (SIFT) algorithm for feature detection and matching, so you will find the [OpenCV-Python documentation](#) on these topics very useful.

The code we provide already implements feature tacking, which identifies feature correspondences between adjacent images. Your job is to implement *feature matching* and allow for inspecting any of the resulting feature matches. This will require coding the following feature matching strategies

- 1) distance thresholding,
- 2) nearest neighbour, and
- 3) nearest neighbour distance ratio.

You will also need to resolve the coordinates of all the features matches involved.

The VO System

Once you refresh the `lab3` branch of your GitLab repo you will find the following Python files.

<code>run_odometry.py</code>	This is the command-line tool that runs the visual odometry according to the subcommand (and the parameters) provided by the user. It contains the <code>RunOdometry</code> class.
<code>visual_odometry.py</code>	This is the main module of the visual odometry system implementing the <code>PinholeCamera</code> and <code>VisualOdometry</code> classes.
<code>vision_tasks_base.py</code>	This module contains the <code>VisionTasksBase</code> “abstract” class that specifies the signatures of four methods you need to implement, and implements the interface used in <code>RunOdometry</code> and <code>VisualOdometry</code>. It also contains the <code>DMatch</code> data class, which you need to use instead of the <code>cv2.DMatch</code> data type of OpenCV so that your return values are serialisable.
<code>vision_tasks.py</code>	This is the module that you need to complete for this exercise. It contains the <code>VisionTasks</code> class that is derived from <code>VisionTasksBase</code> , and must implement its abstract methods in order to complete feature matching and retrieve their details.

In order to complete this exercise you will need to understand both `vision_tasks_base.py` and `vision_tasks.py` but you do not need to know the details of how `run_odometry.py` or `visual_odometry.py` are coded.

The VO tool provides comprehensive help messages. To get started run the command

```
$ ./run_odometry.py -h
usage: run_odometry.py [-h] [-d DATASET]
                        {show_drive,show_matches,draw_matches,get_matches} ...

options:
  -h, --help            show this help message and exit
  -d DATASET, --dataset DATASET
                        path to KITTI dataset directory (option required
                        unless dataset is located at ~/MyKITTI)

subcommands:
  select which odometry command to run

{show_drive,show_matches,draw_matches,get_matches}
  show_drive            show the car camera view and the calculated trajectory
  show_matches          show matches for frame feature using OpenCV (and save
                        this image as ./opencv_visual.png)
  draw_matches          draw matches for frame feature using calculated
                        positions (and save this image as ./custom_visual.png)
  get_matches           get calculated positions for matches of frame feature
```

As noted in the introduction, you do not need to specify where the MyKITTI data set is located if you extract it to ~/MyKITTI, otherwise every time you run run_odometry.py you will need to add the appropriate -d option. The VO tool supports four subcommands: show_drive, show_matches, draw_matches and get_matches. Each of these subcommands has its own help message which you can access with commands like

```
$ ./run_odometry.py show_drive -h
usage: run_odometry.py show_drive [-h] [frame_id]

positional arguments:
  frame_id    index of frame to stop visualisation (0 to 500)

options:
  -h, --help  show this help message and exit
```

Thus, if you extracted MyKITTI at /tmp/MyKITTI and want to visualise the trajectory of the AV up to frame 150, you should run the command

```
$ ./run_odometry.py -d /tmp/MyKITTI show_drive 150
vo params: (None, None, '~/MyKITTI')
debug run: show_drive(150,)
Processing frame 150
ret value: None
```

You will see the trajectory and camera view in real time (depending on processing power). The VO tool will open a window that sequentially displays the first 151 images in the data set, with IDs 0 to 150. In another window it will show the true trajectory (drawn in green) and the estimated trajectory (drawn in red) of the vehicle. While it is running you will get progress indication on the command shell showing the frame index being processed. Once the AV drive finishes the OpenCV GUI will stay open until you press a key to quit the visualisation.

🔑 You need to use a graphical environment that the OpenCV library supports, in order to run the show_drive, show_matches and draw_matches subcommands. These will work as expected if you use the Ubuntu graphical login on Kilburn lab machines.

Determining the vehicle's trajectory only involves feature tracking, which has been implemented for you in `VisionTasksBase`. Once you start implementing the feature matching algorithms, you can begin to use the subcommands to show and report on feature matches. You will probably use `show_matches` first; to get the command-line help run

```
$ ./run_odometry.py show_matches -h
usage: run_odometry.py show_matches [-h] -a {dt,nn,nndr} -t THRESHOLD
                                   frame_id feature_id

positional arguments:
  frame_id              index of chosen frame (0 to 499)
  feature_id            index of chosen feature (0 to 1499)

options:
  -h, --help            show this help message and exit
  -a {dt,nn,nndr}, --algorithm {dt,nn,nndr}
                        name of the algorithm used for feature matching
  -t THRESHOLD, --threshold THRESHOLD
                        decimal value of threshold for feature matching
                        (option not required for nn algorithm)
```

The subcommands `draw_matches` and `get_matches` have similar command-line options.

The choices of algorithm correspond to the feature matching strategies that you will implement in Tasks 1 to 3 below. These are

`dt` to use your implementation of *distance thresholding*,
`nn` to use your implementation of *nearest neighbour*, and
`nndr` to use your implementation of *nearest neighbours distance ratio*.

The valid ranges of threshold value depend on the choice of matching algorithm, as follows

- between 0 and 500 (inclusive¹) for the `dt` and `nn`,
- between 0 and 1 (inclusive) for the `nndr` algorithm.

The threshold value is optional when using `nn` as described in Task 2.

Both feature tracking and feature matching depend on feature detection, which in our VO system is based on the [Scale-Invariant Feature Transform \(SIFT\)](#) algorithm mentioned above. The required functionality is implemented in `VisionTasksBase` by wrapping the `detectAndCompute` method of the `cv2.SIFT` class of OpenCV.

Each matching algorithm will try to find matches for a chosen feature of a given frame in the next frame, which is why the maximum frame index for the feature matching subcommands is 499 and not 500. Each frame will have features with IDs between 0 and 1499 because `VisionTasksBase` is configured to extract *at most* 1500 feature [keypoints](#) and descriptors from each image.

As an example, once you completed Task 1, you can view the matches of feature 1370 of frame 100 according to the distance thresholding algorithm for a threshold value of 232.5 by running

```
$ ./run_odometry.py show_matches -a dt -t 232.5 100 1370
vo params: ('dt', 232.5, '~/MyKITTI')
debug run: show_matches(100, 1370)
Processing frame 101
ret value: None
```

1. As this value is applied on Euclidean distances, in principle the threshold can be any positive number. However, for this exercise, only threshold values between 0 and 500 (inclusive) will be tested.

This will display frames 100 and 101 side-by-side and OpenCV, using its own `cv2.drawMatchesKnn` function, will draw lines from feature 1370 in the first frame to its matches in the second, as shown in Figure 1.

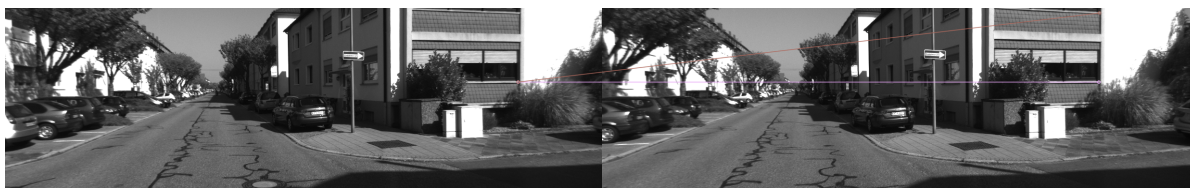


Figure 1: Viewing feature matches

Because of the small size of Figure 1, it is difficult to see that feature 1370 is the bottom right corner of the lower window in frame 100, and that it is matched with the bottom right corners of the upper and lower windows in frame 101, but this will be easier to visualise on the screen.

The SIFT algorithm generates indices for the keypoints that it detects. This happens at runtime and **the numbering will vary** with the versions of OpenCV and the C libraries it depends on. This means that keypoint with index 127 on one machine may have index 132 on another (as happened during testing of the VO system for feature 1370 shown above). It follows that the numbering of image features cannot use keypoint indices for the VO system to have consistent results across different platforms.

- ✧ For this lab exercise **feature IDs** will be numbered in ascending order of the corresponding keypoint coordinates. Your implementation of the feature matching algorithms needs to return matches **sorted in this feature order**.

When you first attempt the tasks of this lab exercise you may want to temporarily use keypoint indices as feature indices, and delay implementing feature ID numbering until things are working correctly. If you do this, then the VO system can behave differently depending on the platform it runs on, as noted in the introduction, and you are likely to get **different results to the examples** in this lab manual.

By completing Task 4 below you will add functionality to the VO system and will be able to use the `draw_matches` and `get_matches` subcommands. Using `draw_matches` instead of `show_matches` will produce the image in Figure 2. It shows the location of the selected feature (in green) on the left and the location of its matches (in red) on the right, the coordinates of these will be calculated by your implementation of the `matchLocating` method of the `VisionTasks` class. You can visually check if your code is behaving as expected by comparing the results of these two subcommands.



Figure 2: Viewing location of matches

To get the details of the matches that your function returns, you need to run

```
$ ./run_odometry.py get_matches -a dt -t 232.5 100 1370
vo params: ('dt', 232.5, '~/MyKITTI')
debug run: get_matches(100, 1370)
Processing frame 101
ret value: ((1066, 152), [(1032, 152), (1033, 9)])
ret count: 2
```

As the example shows, this will be a tuple with the following two elements

- a tuple of two integers, giving the x and y coordinates of the chosen feature in the previous frame, and
- a list of tuples, each with x and y coordinates in the current frame matching the chosen feature.

For the example, in frame 100 the bottom right corner of the lower window is located (1066,152) and its matches in frame 101 have coordinates (1032,152) and (1033,9) on the lower and upper window, respectively.

Assignment

This lab exercise requires you to implement three feature matching algorithms and one function for determining the coordinates of the resulting feature matches. The only Python file that you need to modify is `vision_tasks.py`. Stub code has been provided that allows the VO system to run without error, but this code is non-functional and you must modify it to solve the tasks outlined below. The implementation of the `VisionTasks` class that you will develop is referred to as “your solution” in this document.

Your solution will implement the API defined in the `vision_tasks_base.py` module. Although you do not need to make any changes to this file, it is important that you familiarise yourself with its contents before you start developing your solution. In particular, it is useful to note the following details in relation to the `VisionTasksBase` class:

- Its constructor records the choice of matching algorithm & threshold for the VO system by defining the appropriate `matching_algo` method & `threshold` field, respectively. It also creates a `cv2.SIFT` detector that provides its `detectAndCompute` functionality.
- Its `detectAndCompute` method has the same API of [the corresponding function](#) in OpenCV. You should use it to compute a list of keypoints and a list of descriptors for frame images. Note that the maximum size of the list of keypoints and list of descriptors returned is fixed by the `NUM_FEATURES` constant.
- Its `featureMatching` method fixes the API used by the VO system to interact with your implementation of the various feature matching strategies. Your code will be called via `matching_algo` with OpenCV image objects for the frame given on the command-line with a chosen feature and the frame in which matches for this feature need to be found. These correspond to the method arguments `prev_img` and `this_img` respectively.
- Its `matching_algo` method will be a reference to one of the `dt_matching`, `nn_matching`, or `nndr_matching` functions which you’ll implement in `vision_tasks.py` to complete Tasks 1 to 3 below. These functions need to return a list of keypoints for each of the image arguments `prev_img` and `this_img`, as well as a list of matches in the latter for each feature descriptor in the former.
- The `featureMatching` method does not pass the feature ID chosen on the command-line to your `matching_algo` function. Instead, it expects the list of feature matches that your code returns to be sorted in the feature order defined for the VO system in the previous section. The method also assumes that each feature match object that your implementation returns is an instance of the `DMatch` class defined in `vision_tasks_base.py`, which it converts (back) to the [cv2.DMatch data type](#) of OpenCV. In case the feature ID exceeds the number of keypoints that `detectAndCompute` finds, the method simply returns an empty list of matches.

The course lectures covered the distance thresholding, nearest neighbour, and nearest neighbour distance ratio strategies that your solution needs to implement for the feature matching options `dt`, `nn`, and `nndr` of the VO system to work. In addition, you will find that the [OpenCV-Python tutorial on feature matching](#) includes very useful examples covering technical details required for your implementation. We highlight some of these below.

Your `VisionTasks` class should use the Brute-Force matcher implemented by the `cv2.BFMatcher` class of OpenCV. This class provides methods for computing the similarity, in terms of distance, between image feature descriptors. When instantiated with default parameters

```
bf = cv2.BFMatcher()
```

the Euclidean distance is used as the distance metric, which is what is required for the VO system. Once keypoints and their descriptors have been computed for a pair of images, the `knnMatch` method of the `cv2.DescriptorMatcher` API to compute matches between two sets of feature descriptors. For example

```
matches = bf.knnMatch(des1, des2, k=100)
```

will return the closest 100 matches in the *train image* for every descriptor in the *query image*, sorted by increasing Euclidean distance. The arguments `des1` and `des2` provide the descriptors for the query image and train image, respectively, which in your solution should correspond to the previous and current frames of the VO system. Meanwhile, the value `k` specifies the maximum number of closest matches that the Brute-Force matcher be return for each query descriptor.

The above call to `knnMatch` will return a list `matches` which is of the same size as the number of descriptors in the query image. Each element in `matches` is a list itself that contains `k` elements which are of type `cv2.DMatch`. The feature matching functions that you are required to implement need to return matches using a similar type: a list of lists of `DMatch` objects. The two important differences are that your code needs to filter out matches in accordance with the specified algorithm & threshold value, and it needs to convert the non-serialisable `cv2.DMatch` objects to instances of the `DMatch` class defined in `vision_tasks_base.py`.

To complete the Task 4 given below, you will need to use the information that `DMatch` objects provide to calculate the image coordinates of the corresponding features. Every `DMatch` object `m` contains the indices of the keypoints in the query and train images which have been considered as a match, which can be obtained using

```
query_kp_index = m.queryIdx  
train_kp_index = m.trainIdx
```

Note that this retrieves integers in the runtime generated keypoint numbering, **and not** in the feature ID numbering required for the VO system. If `kp1` and `kp2` are the list of keypoints computed for the query image and train image, respectively, then the keypoints involved in the match `m` are

```
query_kp = kp1[query_kp_index]  
train_kp = kp2[train_kp_index]
```

Here, each of `query_kp` and `train_kp` is of type `cv2.KeyPoint`. These objects have an attribute called `pt` from which the `x` and `y` coordinates of the keypoint in the corresponding image can be determined.

Bearing in mind the above notes on OpenCV functions and classes, you can proceed to completing the tasks below.

Task 1: In your solution, write a function called `dt_matching` that using the *distance thresholding* algorithm implements the `featureMatching` API described at the beginning of this section. It must return feature matches as a list of lists of `DMatch` objects.

You can verify that your function behaves correctly using the VO system tool. For example, once you complete Task 4 you should obtain the following output.

```
$ ./run_odometry.py get_matches -a dt 120 723 -t 150
vo params: ('dt', 150.0, '~/MyKITTI')
debug run: get_matches(120, 723)
Processing frame 121
ret value: ((493, 277), [(458, 288)])
ret count: 2
$ ./run_odometry.py get_matches -a dt 120 723 -t 190
vo params: ('dt', 190.0, '~/MyKITTI')
debug run: get_matches(120, 723)
Processing frame 121
ret value: ((493, 277), [(458, 288), (441, 239), (594, 13)])
ret count: 2
$ ./run_odometry.py get_matches -a dt 120 723 -t 200
vo params: ('dt', 200.0, '~/MyKITTI')
debug run: get_matches(120, 723)
Processing frame 121
ret value: ((493, 277), [(458, 288), (441, 239), (594, 13), (785, 346), (760, 346)])
ret count: 2
```

Task 2: In your solution, write a function called `nn_matching` that using the *nearest neighbours* algorithm implements the `featureMatching` API described at the beginning of this section. It must return feature matches as a list of lists of `DMatch` objects.

Recall that the nearest neighbours algorithm can be used with or without a threshold value (hence why the `--threshold` argument is optional if `--algorithm` is set to `nn`) and your function must behave accordingly.

Again, you can verify that your function behaves correctly using the VO system tool. For example, once you complete Task 4 you should obtain the following output.

```
$ ./run_odometry.py get_matches -a nn 120 723
vo params: ('nn', None, '~/MyKITTI')
debug run: get_matches(120, 723)
Processing frame 121
ret value: ((493, 277), [(458, 288)])
ret count: 2
$ ./run_odometry.py get_matches -a nn 120 723 -t 150
vo params: ('nn', 150.0, '~/MyKITTI')
debug run: get_matches(120, 723)
Processing frame 121
ret value: ((493, 277), [(458, 288)])
ret count: 2
$ ./run_odometry.py get_matches -a nn 120 723 -t 50
vo params: ('nn', 50.0, '~/MyKITTI')
debug run: get_matches(120, 723)
Processing frame 121
ret value: ((493, 277), [])
ret count: 2
```


Task 3: In your solution, write a function called `nndr_matching` that using the *nearest neighbours distance ratio* algorithm implements the `featureMatching` API described at the beginning of this section. It must return feature matches as a list of lists of `DMatch` objects.

As above, you can verify that your function behaves correctly using the VO system tool. For example, once you complete Task 4 you should obtain the following output.

```
$ ./run_odometry.py get_matches -a nndr 120 723 -t .5
vo params: ('nndr', 0.5, '~/MyKITTI')
debug run: get_matches(120, 723)
Processing frame 121
ret value: ((493, 277), [(458, 288)])
ret count: 2
$ ./run_odometry.py get_matches -a nndr 120 723 -t .2
vo params: ('nndr', 0.2, '~/MyKITTI')
debug run: get_matches(120, 723)
Processing frame 121
ret value: ((493, 277), [])
ret count: 2
```

Task 4: In your solution, write a function called `matchLocating` that given OpenCV image objects for a chosen frame and its immediate successor, as well as a specified image feature ID, determines the coordinates of the feature in the chosen image and those of every match in the successor image as computed by the selected matching algorithm for the VO system. Its return value must be a tuple with the following two elements

- a tuple of two integers giving the *x* and *y* image coordinates of the chosen feature, and
- a list of tuples, each with *x* and *y* image coordinates of its matches.

Note that the coordinates of the chosen feature need to be calculated even when the list of matches is empty. In case the feature ID exceeds the number of keypoints for the chosen frame, the coordinates of the feature should be fixed at (0,0).

You can verify that your function behaves correctly using the VO system tool by inspecting the return values of the `get_matches` examples given for Tasks 1, 2 and 3 above. In addition, you should obtain the following output.

```
$ ./run_odometry.py get_matches -a dt -t 232.5 270 1370
vo params: ('dt', 232.5, '~/MyKITTI')
debug run: get_matches(270, 1370)
Processing frame 271
ret value: ((0, 0), [])
ret count: 2
```

For non-trivial examples, you should also use the `draw_matches` subcommand to visualise the coordinates that you have obtained, and check these are the same as those that the `show_matches` subcommand displays using the `cv2.drawMatchesKnn` functionality of OpenCV.

Submission

Please follow the `README.md` instructions in your `COMP24011_2024` GitLab repo. Refresh the files of your `lab3` branch and develop your solution to the lab exercise. The solution consists of a single file called `vision_tasks.py` which must be committed to your GitLab repo and tagged as `lab3_sol`. The `README.md` instructions that accompany the lab files include the `git` commands necessary to commit, tag, and then push **both** the commit and the tag to your `COMP24011_2024` GitLab repo. Further instructions on coursework submission using GitLab can be found in Appendix L of the [CS Handbook](#), including how to change a `git` tag after pushing it.

The deadline for submission is **09:00 on Monday 25th November**. In addition, no work will be considered for assessment and/or feedback if submitted more than **1 week** after the deadline. (Of course, these rules will be subject to any mitigating procedures that you have in place.)

The lab exercise will be **auto-marked** offline. The automarker program will download your submission from GitLab and test it against our reference implementation. For each task the return value of your function will be checked on a random set of valid arguments. A time limit of 10 seconds will be imposed on every function call, and exceeding this time limit will count as a runtime error. If your function does not return values of the correct type, this will also count as a runtime error.

A total of 20 marks is available in this exercise, distributed as shown in the following table.

Task	Function	Marks
1	<code>VisionTasks.dt_matching()</code>	5
2	<code>VisionTasks.nn_matching()</code>	5
3	<code>VisionTasks.nndr_matching()</code>	5
4	<code>VisionTasks.matchLocating()</code>	5

The marking scheme for all tasks is as follows:

- You obtain the first 0.25 marks if all tests complete without runtime errors.
- The proportion of tests with fully correct return values determines the remaining 4.75 marks. The correctness criteria will involve the functional requirements of the VO system's API, as well as the interface specification of its command-line tool.

Important Clarifications

- It will be very difficult for you to circumvent time limits during testing. If you try to do this, the most likely outcome is that the automarker will fail to receive return values from your implementation, which will have the same effect as not completing the call. In any case, an additional time limit of 300 seconds for all tests of each task will be enforced.
- This lab exercise is fully auto-marked. If you submit code which the Python interpreter does not accept, you will score 0 marks. The Python setup of the automarker is the same as the one on the department's Ubuntu image, but only a **minimal** set of Python modules are available. If you choose to add `import` statements to the sample code, it is **your responsibility** to ensure these are part of the default Python package available on the lab machines.
- It doesn't matter how you organise your `lab3` branch, but you should avoid having multiple files with the same name. The automarker will sort your directories alphabetically (more specifically, in ASCII ascending order) and find submission files using breadth-first search. It will mark the first `vision_tasks.py` file it finds and ignore all others.
- Every file in your submission should only contain printable ASCII characters. If you include other Unicode characters, for example by copying and then pasting code from the PDF of the lab manuals, then the automarker is likely to reject your files.